GAZİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ

BM495 BİLGİSAYAR MÜHENDİSLİĞİ PROJESİ I

DÖNEM SONU RAPORU

Ayşe Öz-21118080055

Yusuf Emre Bayrakcı-22118080006

Yazılım Projesi Yönetimi Yazılımı

AKADEMİK DANIŞMAN

Prof. Dr. HACER KARACAN

28.12.2025

4643 Kelime

İÇİNDEKİLER

İÇİNDEKİLER 2
SİMGELER VE KISALTMALAR 4
ÖZET 5
1. GİRİŞ 6
2. LİTERATUR ARAŞTIRMASI 7
2.1. Proje Yönetimi Disiplini ve Temel Bilgi Alanları 7
2.2. Yazılım Araçlarında Bulunan Standart İşlevler 7
2.3. Proje Yönetimi Yazılımlarının Mevcut Durumu 8
2.4. Karşılaşılan Temel Zorluklar 8
2.5. Sektörel Yönelimler ve Gelecek Vizyonu 9
2.6. Literatürden Çıkarımlar ve Projenin Konumu 9
3. GEREKSİNİM ANALİZİ 11
3.1. Kullanıcı Profilleri ve Rol Yapısı 11
3.2. İşlevsel Gereksinimler 11
3.3. İşlevsel Olmayan Gereksinimler (Kalite ve Güvenlik) 13
3.4. Sistem Arayüzleri ve Teknik Altyapı 14
3.5. Proje Yönetimi ve Geliştirme Stratejisi 15
4. SİSTEM TASARIMI 16
4.1. Sistem Mimarisi ve Tasarım Kararları 16
4.2. Yazılım Modülleri ve İç Yapıları 16
4.2.1. Kullanıcı ve Yetkilendirme Modülü (SPMS-MOD-AUTH) : 16
4.2.2. Proje ve Görev Yönetimi Modülü (SPMS-MOD-TASK): 17
4.2.3. Bildirim ve Mesajlaşma Modülü (SPMS-MOD-NOTIF): 17
4.2.4. Raporlama ve Analitik Modülü (SPMS-MOD-REPORT): 17
4.2.5. Süreç Modeli Seçimi ve Özelleştirme Modülü (SPMS-MOD-PROCESS): 18
4.3. Veri Tasarımı (ER Diyagramı ve Varlıklar) 18
4.4. Arayüz Tasarımı ve Entegrasyonlar 18
5. YAPILAN ÇALIŞMALAR 20

5.1. Geliştirme Ortamı ve Veritabanı Altyapısı 20
5.2. Sunucu Tarafı (Backend) Geliştirmeleri 21
5.3. Kullanıcı Arayüzü (Frontend) Geliştirmeleri 21
5.4. Entegrasyon ve Test Süreçleri 22
6. GELECEKTE YAPILACAK ÇALIŞMALAR 24
6.1. Fonksiyonel Modül Genişletmeleri 24
6.2. Teknik Mimari ve Entegrasyon Standartları 25
6.3. Güvenlik, Emniyet ve Uyarlama 25
7. SONUÇ 27
KAYNAKLAR 28

SİMGELER VE KISALTMALAR

Bu çalışmada kullanılmış simgeler ve kısaltmalar, açıklamaları ile birlikte aşağıda sunulmuştur.

Kısaltmalar	**Açıklamalar**
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
GDPR	General Data Protection Regulation
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
PMBOK	Project Management Body of Knowledge
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
UI	User Interface

ÖZET

Yazılım geliştirme süreçlerinin karmaşıklaşması ve ekip içi koordinasyon ihtiyacının artması, modern ve erişilebilir proje yönetim araçlarına olan talebi artırmıştır. Bu proje kapsamında geliştirilen "Yazılım Projesi Yönetim Yazılımı" (SPMS), yazılım ekiplerinin planlama, görev takibi ve iletişim ihtiyaçlarını tek bir platformda karşılamayı hedefleyen web tabanlı bir sistemdir.

Proje, çevik (Agile) yazılım geliştirme prensiplerine dayalı olarak Scrum metodolojisi ile yürütülmektedir. Sistemin mimarisi, sürdürülebilirlik ve performans gereksinimleri gözetilerek modern teknolojiler üzerine inşa edilmiştir. Sunucu tarafında Python (FastAPI) ve PostgreSQL kullanılarak güvenli ve ölçeklenebilir bir arka uç (backend) tasarlanmış; kullanıcı arayüzünde ise React ve TypeScript ile dinamik bir deneyim sunulmuştur.

Bu dönem sonu raporu, projenin birinci geliştirme fazını kapsamaktadır. Dönem içerisinde; sistemin veritabanı mimarisi oluşturulmuş, JWT tabanlı güvenli kimlik doğrulama altyapısı kurulmuş ve temel proje/görev yönetim (CRUD) fonksiyonları başarıyla kodlanmıştır. Yapılan testler, oluşturulan mimarinin Sistem Gereksinim Şartnamesi (SRS) ile uyumlu, güvenli ve genişletilebilir olduğunu doğrulamıştır. Gelecek çalışmalarda, sisteme ileri düzey görselleştirme araçları (Kanban, Gantt) ve gerçek zamanlı bildirim servislerinin entegre edilmesi planlanmaktadır.

1. GİRİŞ

Günümüzde yazılım projelerinin başarısı, sadece teknik yetkinliğe değil, aynı zamanda sürecin ne kadar etkin yönetildiğine bağlıdır. Kapsam kayması, iletişim eksikliği ve kaynakların verimsiz kullanımı gibi riskler, projelerin zamanında teslim edilmesini zorlaştıran temel faktörlerdir. Bu bitirme projesi kapsamında geliştirilen "Yazılım Projesi Yönetim Yazılımı" (SPMS), yazılım geliştirme ekiplerinin proje süreçlerini uçtan uca yönetebilmeleri, görev dağılımlarını organize edebilmeleri ve riskleri minimize edebilmeleri için tasarlanmış web tabanlı bir platformdur.

Projenin temel amacı; görev atamaları, zaman çizelgesi takibi ve ekip içi iletişim gibi kritik proje yönetim fonksiyonlarını modüler bir yapıda sunmaktır. Sistem; Kullanıcı ve Yetkilendirme, Proje ve Görev Yönetimi, Bildirim ve İletişim ile Raporlama ve Analitik olmak üzere dört ana modülden oluşmaktadır. Bu modüler mimari, sistemin esnek, güvenli ve gelecekteki ihtiyaçlara göre genişletilebilir olmasını sağlamaktadır.

Geliştirme sürecinde, değişen gereksinimlere hızlı uyum sağlayabilmek adına Scrum yazılım süreç modeli benimsenmiştir. Teknoloji yığını olarak; arka uçta yüksek performanslı FastAPI, veri katmanında güvenilirliği ile bilinen PostgreSQL ve ön yüzde modern kullanıcı deneyimi sunan React tercih edilmiştir.

Bu rapor; projenin literatürdeki yerini, sistemin gereksinim analizlerini, belirlenen mimari tasarım kararlarını ve dönem boyunca gerçekleştirilen teknik uygulamaları (veritabanı tasarımı, backend servisleri, arayüz kodlaması) detaylandırmaktadır. Ayrıca, projenin mevcut durumu ile hedeflenen nihai sürüm arasındaki yol haritası da "Gelecekte Yapılacak Çalışmalar" başlığı altında sunulmuştur.

2. LİTERATUR ARAŞTIRMASI

Bu bölümde, proje yönetimi disiplininin temel dinamikleri, halihazırda piyasada yaygın olarak kullanılan yazılım araçlarının yetkinlikleri ve literatürdeki mevcut boşluklar incelenmiştir. Yapılan tarama, geliştirilecek projenin teorik zeminini oluşturmaktadır.

2.1. Proje Yönetimi Disiplini ve Temel Bilgi Alanları

Literatürde proje yönetimi; kapsam, zaman, maliyet, kalite ve risk gibi kısıtların dengelenmesi süreci olarak tanımlanmaktadır [1]. Uluslararası standart olan PMBOK (Project Management Body of Knowledge) kılavuzuna göre, başarılı bir proje yönetimi yazılımının desteklemesi gereken temel bilgi alanları şunlardır [2]:

- **Kapsam Yönetimi:** Projeye dahil olan ve olmayan işlerin tanımlanması ve değişikliklerin kontrolü.
- **Zaman (Takvim) Yönetimi:** Gantt şemaları veya ağ diyagramları ile faaliyetlerin sıralanması ve süresinin takibi.
- **Maliyet Yönetimi:** Bütçeleme, tahminleme ve harcamaların kontrol altında tutulması.
- **Kaynak Yönetimi:** İnsan kaynağı, ekipman ve materyallerin doğru zamanda doğru işe atanması.
- **Risk Yönetimi:** Belirsizliklerin ve potansiyel sorunların önceden belirlenip yanıt planlarının oluşturulması.

2.2. Yazılım Araçlarında Bulunan Standart İşlevler

Literatür taraması kapsamında incelenen modern proje yönetim araçlarının (Jira, Asana, MS Project vb.), yukarıdaki teorik alanları desteklemek için şu ortak işlevsel modülleri sunduğu tespit edilmiştir [3]:

- **Planlama ve Çizelgeleme:** Görevlerin mantıksal sıraya dizilmesi (Gantt, Timeline).
- **Görev/Sorun Takibi (Issue Tracking):** İşlerin sorumlulara atanması ve durumlarının (yapılacak, devam ediyor, bitti) güncellenmesi.

- **Doküman ve Dosya Yönetimi:** Teknik çizimlerin ve gereksinimlerin merkezi bir depoda versiyonlanarak saklanması.
- **İletişim ve İşbirliği:** E-posta trafiğini azaltmak için görev kartları üzerinde yorumlaşma ve anlık bildirimler [4].
- **Raporlama ve Gösterge Tabloları (Dashboards):** Proje sağlığının anlık metriklerle görselleştirilmesi [5].

2.3. Proje Yönetimi Yazılımlarının Mevcut Durumu

Literatürde proje yönetimi; kapsam, zaman, maliyet, kalite ve risk gibi kısıtların dengelenmesi süreci olarak tanımlanmaktadır. Bu süreci destekleyen yazılımlar, proje başarısını doğrudan etkileyen faktörler arasındadır. Mevcut araçlar incelendiğinde üç ana kategori öne çıkmaktadır:

- **Kapsamlı ve Geleneksel Araçlar:** Microsoft Project gibi yazılımlar, Şelale (Waterfall) yöntemi, Gantt şemaları ve kaynak yönetimi konularında çok güçlüdür [6]. Ancak dik öğrenme eğrileri ve karmaşık yapıları nedeniyle küçük ekipler veya teknik olmayan kullanıcılar için zorlayıcı bulunmuştur [7].
- **Çevik (Agile) Odaklı Araçlar:** Jira, yazılım geliştirme süreçlerinde (Scrum/Kanban) endüstri standardı olsa da, terminolojisinin yazılım dışı sektörlere yabancı gelmesi ve kullanım zorluğu bir dezavantaj olarak öne çıkmaktadır [7][8]. Trello ise kullanım kolaylığı ve görsel panoları ile bilinmekle birlikte, karmaşık projelerde raporlama ve bağımlılık takibi konularında yetersiz kalmaktadır [9].
- **Açık Kaynak ve Hibrit Araçlar:** OpenProject gibi araçlar lisans maliyeti avantajı sunarken, kullanıcı arayüzü ve deneyimi açısından ticari rakiplerine göre daha az modern bulunabilmektedir [10]. Asana ise esnekliği ile öne çıkmakta ancak büyük ölçekli kullanımlarda maliyet etkinliğini yitirebilmektedir [11].

2.4. Karşılaşılan Temel Zorluklar

Literatür taraması, sadece teknik değil, organizasyonel zorlukların da proje başarısını etkilediğini ortaya koymuştur [3][7]:

- **Doğru Aracı Seçme Problemi:** Kullanıcılar genellikle özellik zenginliği ile kullanım kolaylığı arasında bir seçim yapmak zorunda kalmakta; proje tipine uymayan araç seçimi başarısızlığa yol açmaktadır.
- **Entegrasyon ve Veri Göçü:** Mevcut sistemlerle (kod depoları vb.) entegrasyon zorlukları ve bir araçtan diğerine geçerken yaşanan veri taşıma problemleri teknik bir engeldir.
- **Süreç Disiplini ve Kültürel Uyum:** En gelişmiş araç bile ekip üyeleri tarafından düzenli güncellenmediğinde (örn. görev durumu girilmediğinde) işlevsiz kalmaktadır. Başarı için aracın teknik özellikleri kadar ekibin disiplini de belirleyicidir.
- **Maliyet:** Kullanıcıların büyük çoğunluğu sınırlı bütçelerle hareket ettiğinden, kullanıcı başına lisanslama modelleri (SaaS) özellikle KOBİ'ler için engel teşkil etmektedir [3].

2.5. Sektörel Yönelimler ve Gelecek Vizyonu

Yazılım proje yönetimi alanındaki akademik çalışmalar ve sektörel raporlar, gelecekteki araçların şekillenmesinde etkili olacak bazı trendleri işaret etmektedir:

- **Yapay Zeka ve Otomasyon:** Gelecekteki sistemlerin, tekrarlayan görevleri otomatize etmesi ve veri analitiğine dayalı öngörüler sunması beklenmektedir [12][13].
- **Hibrit Çalışma Desteği:** Uzaktan çalışma modellerinin yaygınlaşmasıyla birlikte, araçların sadece görev takibi değil, aynı zamanda iletişim ve doküman paylaşımını da içeren bütünleşik bir "dijital çalışma alanı" sunması gerekmektedir.

2.6. Literatürden Çıkarımlar ve Projenin Konumu

Yapılan incelemeler sonucunda, piyasada kullanım kolaylığı ile fonksiyonel derinlik arasında bir denge ihtiyacı olduğu tespit edilmiştir. Literatür taramasının "Sonuç"

bölümünde de belirtildiği üzere, geliştirilecek bir yazılımın sadece mevcut fonksiyonları kopyalaması yeterli değildir.

Bu bağlamda projemiz literatürdeki şu ihtiyaca yanıt vermeyi hedefleyen bir zemin üzerine kurgulanmıştır:

- Kullanıcı dostu bir arayüz ile karmaşık proje yönetim süreçlerini basitleştirmek.
- Yüksek maliyetli ticari yazılımlar ile yetersiz kalan basit araçlar arasında, ölçeklenebilir ve erişilebilir bir alternatif oluşturmak.
- Veri güvenliği ve entegrasyon yeteneklerini ön planda tutan dengeli bir çözüm sunmak.

3. GEREKSİNİM ANALİZİ

Proje kapsamında geliştirilen Yazılım Projesi Yönetim Yazılımı (SPMS), yazılım ekiplerinin süreçlerini planlaması, izlemesi ve raporlaması için tasarlanmış web tabanlı bir sistemdir. Gereksinim analizi sürecinde, sistemin modüler, ölçeklenebilir ve Scrum, Kanban, Waterfall gibi farklı proje yönetim metodolojilerine uyarlanabilir olması hedeflenmiştir.

Aşağıda sistemin kullanıcı rolleri, işlevsel modülleri, teknik gereksinimleri ve geliştirme stratejisi detaylandırılmıştır.

3.1. Kullanıcı Profilleri ve Rol Yapısı

Sistemde veri güvenliğini ve operasyonel hiyerarşiyi sağlamak amacıyla rol bazlı erişim kontrolü (RBAC) benimsenmiştir. Tanımlanan temel aktörler şunlardır:

- **Sistem Yöneticisi:** Kullanıcı hesaplarını yönetir, sistem genel ayarlarını yapılandırır ve rol tanımlamalarını yapar.
- **Proje Yöneticisi:** Yeni projeler oluşturur, ekip üyelerini atar, sprint/görev planlamalarını yapar ve performans raporlarına erişir.
- **Ekip Üyesi:** Kendisine atanan görevleri görüntüler, durumlarını günceller (yapılıyor, tamamlandı vb.), görevlere yorum ekler ve dosya paylaşımında bulunur.
- **Misafir (Kısıtlı Erişim):** İlerleyen sürümlerde eklenecek olan, sisteme sadece salt okunur erişim sağlayan roldür.

3.2. İşlevsel Gereksinimler

Sistemin yetenekleri, yazılım geliştirme yaşam döngüsünün tamamını kapsayacak şekilde beş temel modül altında detaylandırılmıştır. Bu modüller, birbirleriyle entegre çalışarak aşağıdaki fonksiyonları yerine getirir:

1. Kullanıcı ve Yetkilendirme Modülü Sistemin giriş kapısı olan bu modül, güvenli erişimi ve hiyerarşik yönetimi sağlar:

- **Kimlik Doğrulama:** Kullanıcı girişleri JWT (JSON Web Token) standardı ile doğrulanır ve parolalar bcrypt algoritmalarıyla şifrelenerek saklanır.

- **Rol Tabanlı Erişim (RBAC):** Sistemde "Yönetici", "Proje Yöneticisi" ve "Ekip Üyesi" rolleri tanımlıdır. Proje yöneticileri, projelerine özel erişim izinlerini (örneğin "sadece ekibim" veya "belirli kişiler") kendileri yapılandırabilir.
- **Profil ve Ekip Yönetimi:** Kullanıcılar profillerini düzenleyebilir, yeni ekipler oluşturabilir veya mevcut ekiplere üye davet edebilir.

2. Proje ve Görev Yönetimi Modülü Uygulamanın çekirdek işlevselliğini oluşturan bu modülde, görevlerin yaşam döngüsü yönetilir:

- **Görev Yaşam Döngüsü:** Görevler oluşturulabilir, düzenlenebilir, önceliklendirilebilir ve alt görevlere bölünebilir. Benzer içeriğe sahip mükerrer görev girişlerinde sistem kullanıcıyı uyararak gereksiz veri girişini engeller.
- **Gelişmiş Bağımlılıklar:** Görevler arasında "bitmeden-başlayamaz" veya "başlamadan-başlayamaz" gibi mantıksal bağımlılıklar tanımlanabilir, böylece gerçekçi bir iş akışı oluşturulur.
- **Tekrarlayan Görev Yönetimi:** Haftalık toplantılar gibi periyodik etkinlikler için tekrarlayan görevler oluşturulabilir. Sistem, bu görevlerde yapılan bir değişiklikte "sadece bu örneği mi yoksa tüm seriyi mi değiştirmek istiyorsunuz?" sorusunu yönelterek esnek yönetim sağlar.
- **İzleme Görünümleri:** Kullanıcılar görevlerini basit liste, sürükle-bırak destekli Kanban panosu veya zaman çizelgesi üzerinde görüntüleyebilir.

3. Bildirim ve Mesajlaşma Modülü Ekip içi iletişimi merkezileştirmeyi ve e-posta trafiğini azaltmayı hedefler:

- **Çok Kanallı Bildirimler:** Görev atamaları veya durum değişikliklerinde kullanıcılara hem uygulama içi anlık bildirim hem de e-posta gönderilir.
- **Bağlamsal İletişim:** Her görevin kendi içinde bir mesajlaşma/yorum alanı bulunur. Kullanıcılar görev detayından çıkmadan ilgili iş hakkında tartışabilir ve dosya paylaşabilir.
- **Özelleştirilebilir Tercihler:** Kullanıcılar bildirim yoğunluğunu yönetmek için "sessiz mod" veya "sadece önemli olaylar" gibi filtreleme seçeneklerini kullanabilir.

4. Raporlama ve Analitik Modülü Yöneticilere ve ekiplere veri odaklı içgörüler sunar:

- **Görsel Paneller (Dashboards):** Proje ilerleme durumları, görev tamamlanma oranları ve sprint grafikleri görselleştirilerek sunulur.
- **Performans Metrikleri:** Sistem, kullanıcı bazlı "tamamlanan görev sayısı" ve "zamanında teslim oranı" gibi performans metriklerini otomatik olarak hesaplar.
- **Dışa Aktarım:** Hazırlanan raporlar, harici sunumlarda kullanılmak üzere PDF veya Excel formatında dışa aktarılabilir ve filtrelenebilir (kullanıcı, görev, proje bazlı).

5. Süreç Modeli Seçimi ve Özelleştirme Modülü Sistemi rakiplerinden ayıran esnek yapılandırma özelliklerini içerir:

- **Model Şablonları:** Proje oluşturma aşamasında kullanıcı; Scrum, Kanban, Waterfall veya İteratif modellerden birini seçebilir. Sistem, seçilen modele uygun şablonu (örneğin Scrum için Sprint yapısı ve toplantı takvimi) otomatik oluşturur.
- **Özelleştirilebilirlik:** Kullanıcılar, standart şablonlara bağlı kalmak zorunda değildir; aktivite sıralaması, sprint uzunlukları veya toplantı periyotları proje özelinde değiştirilebilir.
- **Modüler Görünümler:** Proje gereksinimlerine göre, mevcut projeye sonradan bir Kanban panosu veya Gantt şeması eklenebilir, böylece hibrit çalışma yöntemleri desteklenir.

3.3. İşlevsel Olmayan Gereksinimler (Kalite ve Güvenlik)

Sistemin sürdürülebilirliği, güvenilirliği ve yasal uyumluluğu için aşağıdaki standartlar belirlenmiştir:

- **Güvenlik:** İstemci ve sunucu arasındaki tüm iletişim HTTPS üzerinden şifrelenir. SQL Enjeksiyonu riskine karşı ORM (Object-Relational Mapping) yapısı kullanılır ve hatalı giriş denemelerinde hesap geçici olarak kilitlenir.
- **Veri İzlenebilirliği (Audit Trail):** Sistemde yapılan kritik değişikliklerin tarihçesi tutulmakta olup, geçmiş veriler versiyonlanarak saklanır. Gerektiğinde değişikliklerin geri alınabilmesi veya incelenebilmesi sağlanarak hesap verebilirlik ilkesi desteklenir.

- **Veri Bütünlüğü ve Uyumluluk:** Kritik veriler için "yumuşak silme" mekanizması uygulanır. Ayrıca sistem, KVKK ve GDPR standartlarına uygun veri işleme politikalarına göre tasarlanmıştır.
- **Performans:** Sistem, maksimum işlemci kullanımını %75'in altında tutacak ve kullanıcı isteklerine ortalama 5 saniyenin altında yanıt verecek şekilde optimize edilmiştir.

3.4. Sistem Arayüzleri ve Teknik Altyapı

Sistem, sürdürülebilirlik ve genişletilebilirlik ilkeleri gözetilerek modern web mimarisi üzerine inşa edilmiştir. Arayüz ve altyapı gereksinimleri üç ana başlıkta özetlenmiştir:

1. Kullanıcı Arayüzü (Frontend)

- **Teknoloji:** Arayüz, React kütüphanesi ve TypeScript kullanılarak geliştirilmiştir.
- **Kullanıcı Deneyimi:** Sistem, masaüstü ve mobil cihazlarla uyumlu (responsive) bir yapıdadır. Görev yönetim panoları sürükle-bırak özelliklerini desteklerken, dashboard ekranları kullanıcının rolüne (Yönetici/Ekip Üyesi) göre dinamik olarak içerik sunacak şekilde tasarlanmıştır.

2. Arka Uç ve API Mimarisi (Backend)

- **Servis Yapısı:** Sunucu tarafı Python (FastAPI) çatısı kullanılarak, RESTful mimariye uygun şekilde yapılandırılmıştır.
- **Standartlar ve Güvenlik:** Tüm servisler JSON formatında haberleşir ve Swagger/Redoc ile otomatik olarak belgelenir. API güvenliği için tüm isteklerde kimlik doğrulama zorunlu tutulmuş ve yetkisiz erişimleri engellemek için hız sınırlandırması uygulanmıştır.

3. Veri Tabanı ve Entegrasyon Altyapısı

- **Veri Yönetimi:** Veri kalıcılığı için PostgreSQL tercih edilmiştir. Veritabanı güvenliğini sağlamak ve SQL enjeksiyon risklerini önlemek amacıyla veritabanı erişimi ORM (SQLAlchemy) katmanı üzerinden soyutlanmıştır.
- **Kurulum ve Genişletilebilirlik:** Uygulama, Docker kullanılarak konteynerize edilmiş, böylece farklı ortamlarda (Test/Prod) kurulum standardizasyonu

sağlanmıştır. Sistem, ileride eklenebilecek harici entegrasyonlar (Slack vb.) için modüler bir altyapıya sahiptir.

3.5. Proje Yönetimi ve Geliştirme Stratejisi

Projenin başarıyla tamamlanması için insan kaynağı ve geliştirme süreci aşağıdaki şekilde yapılandırılmıştır:

- **Proje Ekibi:** Geliştirme süreci, hiyerarşik olmayan iki kişilik bir ekip tarafından yürütülmektedir. Analiz, tasarım, kodlama ve test sorumlulukları ortak paylaşılmakta olup; sürüm kontrolü ve iş takibi Git tabanlı bir ortam üzerinden sağlanmaktadır.
- **Gereksinimlerin Önceliklendirilmesi:** Kaynakların verimli kullanımı adına gereksinimler kritiklik seviyelerine ayrılmıştır. Kimlik doğrulama ve veri güvenliği "1. Seviye (Kritik)" olarak belirlenip önceliklendirilirken; görsel tema değişimi gibi özellikler "4. Seviye (Düşük)" kategorisine alınarak aşamalı bir geliştirme planı izlenmiştir.

4. SİSTEM TASARIMI

Yazılım Projesi Yönetim Yazılımı (SPMS), modern yazılım mühendisliği prensipleri doğrultusunda modüler, ölçeklenebilir ve güvenli bir mimari üzerine inşa edilmiştir. Tasarım süreci, sistemin sadece işlevsel gereksinimleri karşılamasını değil; aynı zamanda sürdürülebilirlik, performans ve güvenlik gibi kalite niteliklerini de en üst düzeyde tutmayı hedeflemiştir.

4.1. Sistem Mimarisi ve Tasarım Kararları

SPMS, gevşek bağlı modüllerden oluşan, olay güdümlü bir mimariye sahiptir. Sistemin teknik omurgasını oluşturan temel tasarım kararları şunlardır:

- **Mimari Yaklaşım:** Sistem, React (Frontend) ve FastAPI (Backend) katmanlarının RESTful API üzerinden haberleştiği istemci-sunucu modeline dayanır. Modüller arası bağımlılığı azaltmak için servis tabanlı bir yapı kurgulanmış ve veri akışında DTO (Data Transfer Object) desenleri kullanılmıştır.
- **Veri Tabanı Stratejisi:** Veri tutarlılığını sağlamak amacıyla ilişkisel bir veritabanı (PostgreSQL) tercih edilmiştir. Performans optimizasyonu için durum ve tarih alanlarında indeksleme yapılmış, kritik verilerin silinmesini önlemek için "soft-delete" mekanizması uygulanmıştır.
- **Gerçek Zamanlı İletişim:** Kullanıcı etkileşimini artırmak adına, bildirim ve mesajlaşma servisleri WebSocket teknolojisi üzerine kurulmuştur. Bu sayede görev güncellemeleri veya mesajlar, sayfa yenilemeye gerek kalmadan kullanıcılara anlık olarak iletilir.

4.2. Yazılım Modülleri ve İç Yapıları

Sistem, her biri belirli bir işlev grubundan sorumlu olan beş ana modülden oluşmaktadır. Her modülün veri yapıları, sınıf tasarımları ve algoritmaları aşağıda detaylandırılmıştır.

4.2.1. Kullanıcı ve Yetkilendirme Modülü (SPMS-MOD-AUTH) : Sistemin güvenlik katmanıdır.

- **Veri Yapısı:** USERS ve ROLES tabloları üzerinden kullanıcı kimliklerini ve yetkilerini yönetir.

- **Algoritma:** Giriş işlemlerinde, kullanıcı parolası veritabanındaki hashlenmiş (bcrypt/argon2) veri ile karşılaştırılır. Doğrulama başarılıysa, kullanıcıya sınırlı süreli bir JWT (JSON Web Token) verilir. Tüm API isteklerinde bu token kontrol edilerek yetkisiz erişimler engellenir.

4.2.2. Proje ve Görev Yönetimi Modülü (SPMS-MOD-TASK): Uygulamanın çekirdek işlevselliğini barındırır.

- **Veri Yapısı:** PROJECTS, TASKS, SPRINTS ve BOARD_COLUMNS tabloları ile projenin iskeletini oluşturur.
- **Dinamik İş Akışı:** Modül, Factory Pattern kullanarak proje oluşturma anında seçilen metodolojiye (Scrum, Kanban vb.) uygun veri yapılarını (sprintler veya pano kolonları) otomatik olarak üretir.
- **Algoritmalar:**
    - **Tekrarlayan Görevler:** Arka planda çalışan bir zamanlayıcı, periyodu gelen görevleri tespit ederek otomatik olarak yeni kopyalarını oluşturur.
    - **Bağımlılık Kontrolü:** Bir görev tamamlanmak istendiğinde, sistem o göreve bağlı başka bir görevin bitip bitmediğini kontrol eder; bitmediyse işlemi reddeder.

4.2.3. Bildirim ve Mesajlaşma Modülü (SPMS-MOD-NOTIF): Sistem içi etkileşimi yönetir.

- **Yapı:** NOTIFICATIONS ve COMMENTS tablolarını kullanır.
- **Olay Güdümlü Algoritma:** Sistemde bir olay (örn: görev ataması) gerçekleştiğinde, modül bu olayı dinler ve ilgili kullanıcının WebSocket kanalına anlık veri paketi gönderir. Kullanıcı çevrimdışı ise bildirim veritabanına "okunmadı" olarak kaydedilir.

4.2.4. Raporlama ve Analitik Modülü (SPMS-MOD-REPORT): Karar destek mekanizmasıdır.

- **Analiz Yöntemi:** Operasyonel tablolardan (TASKS, LOGS) salt okunur (read-only) sorgularla veri çeker.

- **Burndown Chart Algoritması:** Scrum projeleri için, sprintin toplam hikaye puanından tamamlanan işleri gün bazında düşerek kalan iş grafiğini matematiksel olarak hesaplar.
- **Dışa Aktarım:** Hazırlanan rapor verilerini HTML şablonlarına gömerek sunucu tarafında PDF veya Excel dosyasına dönüştürür ve kullanıcıya indirilebilir dosya olarak sunar.

4.2.5. Süreç Modeli Seçimi ve Özelleştirme Modülü (SPMS-MOD-PROCESS):

Sisteme esneklik kazandıran bileşendir.

- **Dinamik Yapı:** Görev durumları kod içinde sabitlenmemiş, veritabanında BOARD_COLUMNS tablosunda dinamik olarak tutulmuştur. Bu sayede her proje kendi iş akışına (Örn: "Analiz -> Test -> Canlı") sahip olabilir.
- **Metodoloji Değişimi:** Kullanıcı çalışma yöntemini değiştirdiğinde (Örn: Scrum'dan Kanban'a geçiş), modül eski verileri (sprintler) koruyarak yeni kuralları (WIP limitleri) aktif hale getirir.

4.3. Veri Tasarımı (ER Diyagramı ve Varlıklar)

Sistemin veri bütünlüğü, kapsamlı bir Varlık-İlişki modeli ile sağlanmıştır. Tasarımda öne çıkan temel ilişkiler şunlardır:

- **Hiyerarşik Görev Yapısı:** TASKS tablosu kendi kendine referans vererek (parent_task_id) alt görev ve bağımlılık ilişkilerini destekler.
- **Esnek Proje Yapısı:** PROJECTS tablosu, metodoloji alanıyla projenin davranışını belirlerken; ROLES tablosu ile kullanıcıların proje üzerindeki yetkileri tanımlanmıştır.
- **İzlenebilirlik (Audit Log):** LOG tablosu, sistemdeki tüm değişikliklerin eski ve yeni değerlerini JSON formatında saklayarak tam bir tarihçe takibi sağlar.

4.4. Arayüz Tasarımı ve Entegrasyonlar

Modüller arası iletişim ve dış dünya ile etkileşim, standartlaştırılmış arayüzler üzerinden yürütülür:

- **API Tasarımı:** Tüm modüller, dış dünyaya RESTful API uç noktaları sunar. Veri alışverişi JSON formatında yapılır ve Swagger ile otomatik belgelenir.

- **Kullanıcı Arayüzü (UI):** React tabanlı arayüz, kullanıcının rolüne göre dinamikleşir. Örneğin, Scrum projesinde "Sprint Planlama" ekranları aktifken, Kanban projesinde bu ekranlar gizlenir.
- **Sistem İçi İletişim:** Modüller birbirine doğrudan bağımlı değildir. Örneğin, bir görev güncellendiğinde Görev Modülü doğrudan Bildirim Modülünü çağırmaz; bir "Olay" yayınlar ve Bildirim Modülü bu olayı yakalar. Bu yapı sistemin bakımını kolaylaştırır.

5. YAPILAN ÇALIŞMALAR

Projenin geliştirme süreci, Sistem Gereksinim Şartnamesi'nde (SRS) tanımlanan isterlere ve Yazılım Tasarım Belgesi'nde (SDD) belirlenen mimari kararlara tam uyum çerçevesinde yürütülmüştür. Bu dönemde; veritabanı şemaları oluşturulmuş, backend servisleri yazılmış, kullanıcı arayüzleri kodlanmış ve Axios kütüphanesi ile uçtan uca güvenli entegrasyon sağlanmıştır.

Aşağıda, gerçekleştirilen çalışmalar teknik detayları ve SRS dokümanındaki karşılıkları ile birlikte sunulmuştur.

5.1. Geliştirme Ortamı ve Veritabanı Altyapısı

Projenin sürdürülebilirliği, veri bütünlüğü ve ekip içi çalışma standartlarını sağlamak amacıyla aşağıdaki altyapı çalışmaları tamamlanmıştır:

- **Sürüm Kontrol ve İş Akışı:** Proje kodları Git deposunda yapılandırılmış; backend ve frontend için modüler dizinler oluşturulmuştur. Geliştirme sürecinde "feature-branch" iş akışı uygulanarak, ana dalın kararlılığı korunmuştur.
    - **Karşılanan İsterler: [SPMS-ENV-3]** (Ortamların ayrıştırılması ve yönetimi), **[SPMS-QLT-5]** (Sürdürülebilirlik ve modülerlik).
- **Konteynerize Veritabanı Kurulumu:** Veritabanı yönetimi için PostgreSQL seçilmiş ve geliştirme ortamları arasındaki tutarlılığı sağlamak amacıyla Docker konteyneri üzerinde yapılandırılmıştır. docker-compose dosyası ile veritabanı servisinin tek komutla, izole bir ortamda ayağa kalkması sağlanmıştır.
    - **Karşılanan İsterler: [SPMS-ENV-2]** (Docker tabanlı konteynerizasyon), **[SPMS-DATA-1]** (İlişkisel veritabanı yapısı).
- **İlişkisel Şema Tasarımı:** SDD'de tasarlanan Varlık-İlişki (ER) modeline sadık kalınarak; USERS, PROJECTS ve TASKS tabloları oluşturulmuştur. Tablolar arası One-to-Many ve Many-to-Many ilişkileri tanımlanmış, veri bütünlüğü için Foreign Key kısıtlamaları ve performans için indekslemeler uygulanmıştır.

- **Karşılanan İsterler: [SPMS-DATA-1]** (Temel tabloların oluşturulması), **[SPMS-DATA-8]** (Veri ilişkileri ve kısıtlamalar), **[SPMS-DB-02]** (Tablolar arası ilişkilerin tanımlanması).

5.2. Sunucu Tarafı (Backend) Geliştirmeleri

Sistemin iş mantığını yürüten servisler Python (FastAPI) ile geliştirilmiş olup, güvenlik ve veri yönetimi modülleri aşağıdaki gibi tamamlanmıştır:

- **API Mimarisi ve Dokümantasyon:** RESTful mimariye uygun olarak modüler bir API iskeleti (Auth, Task, Project router'ları) kurulmuştur. Geliştirici deneyimini artırmak için Swagger UI entegrasyonu yapılmış, tüm uç noktaların canlı dokümantasyonu sağlanmıştır.
    - **Karşılanan İsterler: [SPMS-API-03]** (REST standartları), **[SPMS-API-05]** (Otomatik dokümantasyon).
- **Kimlik Doğrulama (Auth) Servisi:** Güvenli oturum yönetimi için JWT (JSON Web Token) tabanlı kimlik doğrulama mekanizması kodlanmıştır. Kullanıcı kayıt (/register) ve giriş (/login) işlemleri yazılmış, parolalar veritabanına kaydedilmeden önce bcrypt algoritması ile hashlenerek güvenlik altına alınmıştır.
    - **Karşılanan İsterler: [SPMS-01.1]** (Güvenli giriş/çıkış), **[SPMS-01.3]** (JWT kullanımı), **[SPMS-SEC-01]** (JWT mekanizması), **[SPMS-SEC-02]** (Parola şifreleme).
- **Temel CRUD Servisleri:** Proje ve görev yönetimi için gerekli olan Oluşturma, Okuma, Güncelleme ve Silme (CRUD) servisleri yazılmıştır. Bu servisler, veritabanı ile SQLAlchemy ORM katmanı üzerinden güvenli bir şekilde haberleşmektedir.
    - **Karşılanan İsterler: [SPMS-02.1]** (Proje işlemleri), **[SPMS-02.3]** (Görev işlemleri), **[SPMS-DB-01]** (ORM kullanımı).

5.3. Kullanıcı Arayüzü (Frontend) Geliştirmeleri

Ön yüz geliştirmelerinde React ve TypeScript kullanılarak, kullanıcı rollerine göre özelleşmiş dinamik arayüzler kodlanmıştır:

- **Proje Yöneticisi Dashboard'u:** Yöneticilerin projelerini genel hatlarıyla izleyebildiği özet panel tasarlanmıştır. Kullanıcı rolüne göre içerik dinamik olarak değişmektedir (Örn: Yönetici istatistikleri görürken, çalışan sadece görevleri görür).
    - **Karşılanan İster: [SPMS-UI-01]** (Rol bazlı dashboard özelleştirme).
- **Çalışan Odaklı "Görevlerim" Ekranı:** Ekip üyelerinin sadece kendilerine atanan görevleri görebildiği kişisel çalışma alanı geliştirilmiştir. Bu ekran, yetki kontrolü mantığıyla çalışmakta ve kullanıcının erişim izni olmayan projeleri filtrelemektedir.
    - **Karşılanan İster: [SPMS-01.5]** (Proje erişim izinleri).
- **Görev Durum Güncelleme Etkileşimi:** Kullanıcıların görev statülerini (Yapılacaklar -> Tamamlandı) hızlıca değiştirebilmesi için görev kartlarına durum butonları entegre edilmiştir. Arayüz bileşenleri modüler tasarlanarak (Component-based) kod tekrarı önlenmiştir.
    - **Karşılanan İsterler: [SPMS-02.4]** (Görev durum yönetimi), **[SPMS-UI-05]** (Yeniden kullanılabilir UI bileşenleri).

5.4. Entegrasyon ve Test Süreçleri

Geliştirilen modüllerin uyumlu ve güvenli çalışması için Axios kütüphanesi ile entegrasyon sağlanmış ve güvenlik testleri gerçekleştirilmiştir:

- **Axios ile Güvenli API Entegrasyonu:** React uygulamasının backend ile haberleşmesi için Axios HTTP istemcisi tercih edilmiştir. Axios'un "Interceptors" özelliği yapılandırılarak, kullanıcının aldığı JWT token'ın her API isteğine otomatik olarak Authorization: Bearer <token> başlığıyla eklenmesi sağlanmıştır. Bu sayede tüm uç noktalarda kimlik doğrulama garanti altına alınmıştır.
    - **Karşılanan İsterler: [SPMS-API-01]** (Tüm uç noktalarda kimlik doğrulama), **[SPMS-SEC-03]** (Güvenli veri iletişimi).
- **Rol Bazlı Erişim Kontrolü (RBAC) Testleri:** Sistemin güvenlik katmanını doğrulamak amacıyla Pytest kullanılarak backend birim testleri yazılmıştır. Bu testler ile; "Yönetici" yetkisine sahip olmayan bir kullanıcının proje silme veya başkasının görevini değiştirme işlemlerinin sunucu tarafında reddedildiği (HTTP 403 Forbidden) doğrulanmıştır.

- **Karşılanan İsterler: [SPMS-01.2]** (Rol bazlı erişim), **[SPMS-SEC-04]** (Sunucu tarafı katı yetki kontrolü), **[SPMS-QLT-7]** (Test edilebilirlik).

6. GELECEKTE YAPILACAK ÇALIŞMALAR

Projenin ilk geliştirme fazında (1.Dönem), sistemin omurgasını oluşturan veritabanı mimarisi, temel kimlik doğrulama servisleri ve çekirdek CRUD (Veri Oluşturma/Okuma) işlemleri tamamlanmıştır. Önümüzdeki geliştirme döneminde (2.Dönem), Sistem Gereksinim Şartnamesi'nde (SRS) taahhüt edilen ancak henüz implementasyonu tamamlanmamış tüm modüllerin hayata geçirilmesi hedeflenmektedir.

Planlanan çalışmalar; Fonksiyonel Geliştirmeler, Teknik Altyapı Standartları ve Sistem Güvenliği/Kalitesi olmak üzere üç ana kategoride toplanmıştır.

6.1. Fonksiyonel Modül Genişletmeleri

Sistemin sadece veri tutan bir yapıdan, süreç yöneten akıllı bir platforma dönüşmesi için aşağıdaki fonksiyonel paketler geliştirilecektir:

- **Süreç Modeli ve Görselleştirme Altyapısı:** Mevcut liste görünümüne ek olarak, SRS'de tanımlanan farklı proje yönetim metodolojileri sisteme entegre edilecektir. Kullanıcıların Scrum, Kanban veya Waterfall modellerini seçebilmesi sağlanacak ve buna bağlı olarak sürükle-bırak destekli Kanban Panoları , zaman eksenli Gantt Şemaları ve takvim görünümleri geliştirilecektir. Bu geliştirmeler, ilgili arayüz gereksinimleri ile uyumlu olacaktır.
- **İleri Düzey Görev Yönetimi:** Görev modülü, basit statü takibinin ötesine taşınacaktır. Görevler arası "Başlamadan-Başlayamaz" gibi mantıksal bağımlılıklar tanımlanacak , haftalık/aylık tekrarlayan görev motoru yazılacak ve benzer görev girişlerini önleyen uyarı mekanizmaları eklenecektir. Ayrıca görev detaylarında dosya paylaşımı ve yorumlaşma özellikleri aktif edilecektir .
- **Bildirim ve İletişim Merkezi:** Kullanıcıların sistemdeki olaylardan (görev atama, güncelleme vb.) anında haberdar olması için gerçek zamanlı bildirim servisi kurulacaktır. Bu servis, SRS'de belirtildiği üzere hem uygulama içi (WebSocket) hem de E-posta kanallarını destekleyecek ve kullanıcıların bildirim tercihlerini özelleştirmesine izin verecektir .
- **Raporlama ve Analitik:** Yöneticiler için karar destek mekanizması olarak çalışacak raporlama modülü devreye alınacaktır. Takım performansı, görev tamamlama

oranları ve sprint ilerleme grafikleri görselleştirilecek; oluşturulan raporların PDF/Excel formatında dışa aktarımı sağlanacaktır .

6.2. Teknik Mimari ve Entegrasyon Standartları

Sistemin arka planında çalışan servislerin, SRS'de belirtilen teknik kısıt ve standartlara tam uyumu sağlanacaktır:

- **API ve Veri İletişim Standartları:** Backend servislerinde SRS Bölüm 1.3.3'te belirtilen standartlar sıkılaştırılacaktır. Hatalı istekler için standart HTTP hata kodları (400, 401, 403, 500) yapılandırılacak , kötüye kullanımı önlemek için hız sınırlandırması uygulanacak ve katı CORS politikaları ile sadece güvenilir alan adlarına izin verilecektir .
- **Veritabanı Bütünlüğü ve Yönetimi:** Veri katmanında, SRS Bölüm 1.5 gerekliliklerine uygun olarak "Yumuşak Silme" mekanizması kurulacak (veriler kalıcı silinmeyecek, pasife alınacak). Tüm kritik tablolarda sürümleme ve tarihçe takibi altyapısı oluşturularak verilerin geçmişe dönük izlenebilirliği sağlanacaktır .
- **Dış Sistem Entegrasyonları:** Projenin genişletilebilirlik vizyonu çerçevesinde; Google Calendar, Slack veya MS Teams gibi üçüncü parti uygulamalarla entegrasyon sağlayacak servis katmanları hazırlanacaktır . Bu entegrasyonlarda OAuth token yönetimi güvenli bir şekilde sağlanacaktır .

6.3. Güvenlik, Emniyet ve Uyarlama

Son kullanıcı deneyimini ve sistem güvenliğini en üst seviyeye taşımak için aşağıdaki "Niteliksel Gereksinimler" hayata geçirilecektir:

- **İleri Güvenlik Önlemleri:** Mevcut JWT yapısına ek olarak, SRS Bölüm 1.8'de belirtilen; parola sıfırlama süreçleri (token tabanlı e-posta linki) ve Kaba Kuvvet saldırılarına karşı hesap kilitleme mekanizmaları implemente edilecektir . Veri gizliliği kapsamında KVKK/GDPR uyumluluğu gözetilecektir .
- **Emniyet (Safety) Protokolleri:** Kullanıcı hatalarını ve yetkisiz erişimi önlemek adına; kritik işlemlerde (proje silme vb.) "Onay Penceresi" zorunluluğu getirilecek ve hareketsiz kalan oturumlar için otomatik zaman aşımı kuralı işletilecektir .
- **Uyarlama ve Konfigürasyon:** Sistemin farklı organizasyonlara uyum sağlayabilmesi için Bölüm 1.6'daki uyarlama gereksinimleri kodlanacaktır.

Yöneticilerin sistem parametrelerini (tema, varsayılan ayarlar, aktif modüller) kod değişikliği yapmadan değiştirebileceği yönetim panelleri geliştirilecektir .

- **Yazılım Kalite Güvencesi:** Sistemin kararlılığını garanti altına almak için SRS Bölüm 1.11'de hedeflenen kalite faktörlerine odaklanılacaktır. Özellikle ortalama yanıt süresinin 5 saniyenin altında tutulması için sorgu optimizasyonları yapılacak ve CI/CD süreçlerine entegre edilecek uçtan uca test senaryoları yazılacaktır .

7. SONUÇ

2025-2026 Güz döneminde yürütülen "Yazılım Projesi Yönetim Yazılımı" projesinin birinci faz çalışmaları, belirlenen proje takvimi doğrultusunda yürütülmüş ve temel geliştirme hedeflerine büyük ölçüde ulaşılmıştır. Sistem Gereksinim Şartnamesi'nde (SRS) tanımlanan altyapı gereksinimleri ve Yazılım Tasarım Belgesi'nde (SDD) kurgulanan mimari yapı referans alınarak, sistemin temel prototipi ortaya konulmuştur.

Bu süreçte gerçekleştirilen çalışmalar ve ulaşılan aşamalar şu şekildedir:

- **Altyapı Kurulumu:** Docker üzerinde konteynerize edilmiş PostgreSQL veritabanı ve modüler FastAPI mimarisi ile sistemin teknik temeli yapılandırılmıştır.
- **Güvenlik:** JWT tabanlı kimlik doğrulama ve Rol Bazlı Erişim Kontrolü (RBAC) mekanizmaları sunucu tarafına entegre edilerek, temel veri güvenliği standartlarının sağlanması hedeflenmiştir.
- **İşlevsellik:** Kullanıcıların proje oluşturma ve görev atama gibi çekirdek işlevleri (CRUD) gerçekleştirebildiği, React tabanlı bir arayüz geliştirilmiştir.
- **Entegrasyon:** Frontend ve Backend servisleri arasındaki veri akışı, Axios kütüphanesi kullanılarak işlevsel hale getirilmiştir.

Gerçekleştirilen birim ve entegrasyon testleri sonucunda, sistemin temel fonksiyonlarının beklenen senaryolar dahilinde çalıştığı gözlemlenmiştir. Bu dönem ortaya konan çalışma, projenin nihai sürümü için bir zemin niteliği taşımaktadır. Elde edilen bu temel yapı (MVP) üzerine, önümüzdeki dönemde Kanban, Gantt, gerçek zamanlı bildirimler ve raporlama modüllerinin eklenmesiyle projenin kapsamının genişletilmesi planlanmaktadır.

KAYNAKLAR

[1] Project Management Institute (PMI), "The Triple Constraint: Erroneous, Useless, or Value?," PMI. [Çevrimiçi]. Erişim: https://www.pmi.org/learning/library/triple-constraint-erroneous-useless-value-8024 Erişim tarihi: 26 Aralık 2025.

[2] Project Management Institute (PMI), *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 6. bs., 2017.

[3] Y. Chouhan, A. Sangle, A. Patil, S. Ramteke ve K. V. Metre, "Project Management Tool: A Review," *International Journal of Scientific Development and Research*, cilt 7, sayı 1, ss. 44-49, 2022. [Çevrimiçi]. Erişim: https://www.ijsdr.org/papers/IJSDR2201044.pdf Erişim tarihi: 26 Aralık 2025.

[4] Aprika, "Project management tools," Aprika. [Çevrimiçi]. Erişim: https://aprika.com/fundamental-library/project-management-tools/ Erişim tarihi: 26 Aralık 2025.

[5] Capterra, "Project Management Software User Research," Mar. 2021. [Çevrimiçi]. Erişim: https://www.capterra.com/resources/project-management-software-user-research/ Erişim tarihi: 26 Aralık 2025.

[6] Microsoft, "Microsoft Planner," [Çevrimiçi]. Erişim: https://www.microsoft.com/tr-tr/microsoft-365/planner/microsoft-planner Erişim tarihi: 26 Aralık 2025.

[7] M. A. Miah, C. R. Barikdar, H. Rahman, F. Mahmud ve J. Alam, "Comparative Analysis of Project Management Software: Functionality, Usability, and Integration for Modern Workflows," *Membrane Technology*, cilt 2025, sayı 1, 2025. [Çevrimiçi]. Erişim: https://membranetechnology.org/index.php/journal/article/download/309/215/607

[8] Atlassian, "Jira," [Çevrimiçi]. Erişim: https://www.atlassian.com/software/jira Erişim tarihi: 26 Aralık 2025.

[9] Trello, "Trello," [Çevrimiçi]. Erişim: https://trello.com/tr Erişim tarihi: 26 Aralık 2025.

[10] OpenProject, "OpenProject," [Çevrimiçi]. Erişim: https://www.openproject.org/ Erişim tarihi: 26 Aralık 2025.

[11] Asana, "Asana," [Çevrimiçi]. Erişim: https://asana.com/ Erişim tarihi: 26 Aralık 2025.

[12] Artsmart.ai, "AI in Project Management: 2025 Trends, Stats, and Future Outlook," Artsmart Blog, 2023. [Çevrimiçi]. Erişim: https://artsmart.ai/blog/ai-in-project-management-statistics/ Erişim tarihi: 26 Aralık 2025.

[13] "Rise of Artificial Intelligence in Project Management: A Systematic Literature Review of Current Opportunities, Enablers, and Barriers," *Information*, 15(7), 1130. [Çevrimiçi]. Erişim: https://www.mdpi.com/2075-5309/15/7/1130 Erişim tarihi: 26 Aralık 2025.

İNTİHAL BEYANI

Bu çalışmadaki tüm bilgilerin akademik kurallara ve etik davranışa uygun olarak alındığını ve sunulduğunu ve bu belgede alıntı yaptığımı belirttiğim yerler dışında sunduğum çalışmanın kendi çalışmam olduğunu, Yükseköğretim Kurumları Bilimsel Araştırma Ve Yayın Etiği Yönergesinde belirtilen bilimsel araştırma ve yayın etiği ilkelerine uygun olduğunu beyan ederim.

Numara : 21118080055 22118080006

Ad Soyad : Ayşe Öz Yusuf Emre Bayrakcı

Tarih : 28/12/2025

İmza :